

Operator Basis Toolkit Report

Arbitrary-dimension linearized graviton operator bases with IBP reduction, Fierz–Pauli field redefinitions, and sector-level caching

This report documents the generalized Mathematica/xAct toolkit that computes scalar operator bases for a linearized graviton up to any chosen maximum mass dimension. It explains the sector-by-sector algorithm, the role of the Euler operator in implementing integration by parts, the quotient by Fierz–Pauli field redefinitions, the caching strategy used for incremental runs, and the path to multi-field generalizations.

Companion Note for graviton_basis_toolkit.wl

Generalized vAct workflow for graviton operator bases, IBP reduction, and Fierz-Pauli field-redefinition reduction

Prepared as a practical guide to the script structure, inputs, outputs, and extension strategy.



This note explains what the toolkit computes sector by sector, how the caching layer is organized, how to interpret the quotient bases, and how to recover the field-redefinitions that remove redundant operators.

Page 1



Page 3

7. Caching behavior

Caching is file-based and sector-local. Intermediate objects such as the raw scalar basis, the rank-2 redefinition basis, and the assembled sector data are stored under deterministic filenames. If you first run up to $D = 6$ and later rerun with $D = 8$ using the same cache directory, all pre-existing sectors are loaded immediately and only the new $D = 8$ sectors are computed. This is especially useful because the $D = 6$ run naturally revisits lower-dimensional sectors unless those cached objects are available.

1. Sector organization and counting conventions

The script works with scalar operator sectors labeled by the number of graviton fields n_g and the total number of derivatives N_d . For a linearized graviton $h_{\mu\nu}$ in four dimensions, the engineering mass dimension is taken to be 1, so the total operator dimension is

$$D = n_g + N_d$$

At fixed (n_g, N_d) the code generates all scalar contractions of products of differentiated fields, canonically simplifies them with vAct, and then performs two quotient operations:

Stage	What it removes	How it is implemented
Raw basis	nothing	All Contractions on differentiated h tensors, followed by canonicalization
IBP quotient	total derivatives	vanishing Euler-Lagrange derivative of a general linear combination
FP quotient	terms removable by field redefinition	changes induced by $h \rightarrow h + \delta h$ inside the quadratic Fierz-Pauli action

For the cubic two-derivative sector, this reproduces the workflow already tested in your notebook: start from the 16 raw $h(\partial)h(\partial)$ structures, quotient by the two IBP relations, and then quotient further by the Fierz-Pauli redefinition image.

2. IBP algorithm

Suppose a raw sector basis is O_i , $i = 1, \dots, N$. The script forms an ansatz

$$L_{\text{ansatz}} = \sum_i c_i O_i$$

and computes the Euler-Lagrange derivative with respect to $h_{\mu\nu}$. A linear combination is IBP trivial precisely when its Euler-Lagrange derivative vanishes identically. Therefore the linear system

$$\delta L_{\text{ansatz}} / \delta h_{\mu\nu} = 0$$

determines all IBP relations among the raw operators. In the code this is done with `VarEulerLagrange`, `+`, `-`, `+`, `+`, `+`. The solution space is converted into a relation matrix, pivot columns are identified, and a quotient basis is built by deleting a pivot representative from each relation class.

3. Fierz-Pauli field-redefinition quotient

Let the free quadratic action be the Fierz-Pauli Lagrangian L_{FP} . A local field redefinition

$$h_{\mu\nu} \rightarrow h_{\mu\nu} + \lambda \delta h_{\mu\nu}$$

changes the action by the first-order shift

$$\Delta L = \text{IBP}(\delta h) L_{\text{FP}} + \lambda \text{FP}(\delta h)$$

For a target scalar sector (n_g, N_d) , the relevant redefinitions have the schematic form $\delta h = (a)^\mu_\nu h^\mu_\nu x^{N_d-1}$. The script constructs a basis of such symmetric rank-2 tensors with free indices, injects them into the shift of L_{FP} , and projects the resulting scalar images onto the IBP quotient basis. The span of those projected vectors is the field-redefinition image. Removing its pivot representatives yields the final physical basis.

Page 2

4. Script structure

Setup

Defines the manifold, flat metric, graviton tensor h , and auxiliary tensors δh and probe .

RawScalarBasis

Builds all scalar contractions in a sector by distributing derivatives among fields and canonicalizing the contractions.

IBPData

Computes the IBP relations by the Euler-Lagrange criterion and returns the relation matrix plus quotient-basis data.

Rank2Basis

Builds the redefinition tensor basis by first generating probe-linear scalars and then varying with respect to the symmetric probe tensor.

ShiftFromRedefExpr

Computes the cubic or higher-order scalar image induced by a given δh insertion into the quadratic FP action.

RedefinitionImageData

Projects all redefinition images into the IBP quotient basis and extracts an independent spanning set.

ComputeSectorData / RunBasisComputation

Assembles sector summaries and loops over all sectors up to the requested maximum dimension.

5. Core usage pattern

```

run = RunBasisComputation()
"RawScalarBasis" -> R,
"IBPData" -> IBP,
"Rank2Basis" -> Rank2,
"ShiftFromRedefExpr" -> Shift,
"RedefinitionImageData" -> Redef,
"ComputeSectorData" -> SectorData,
"RunBasisComputation" -> RunBasisComputation

runSectorSummary[Seed]

physData = PhysData[runSectorData, 1, 2]
redefData = RedefData[runSectorData, 1, 2]
  
```

6. Main outputs to inspect

Accessor	Meaning
<code>RawBasisOfSector[yes, rh, Nd]</code>	All canonical scalar contractions before quotienting
<code>IBPBasisOfSector[yes, rh, Nd]</code>	Basis after quotienting by total derivatives
<code>PhysBasisOfSector[yes, rh, Nd]</code>	Basis after both IBP and FP field redefinitions
<code>RedefImageOfSector[yes, rh, Nd]</code>	Independent δh tensors whose FP images span the removable subspace
<code>RedefImageFluxOfSector[yes, rh, Nd]</code>	Coefficient data expressing removable directions in the IBP basis

Page 4

8. Extending to multi-field systems

The same architecture extends to an EFT with several fields ϕ^a in addition to $h_{\mu\nu}$. The key change is that a sector must now record field content as a multidegree, for example $(n_g, r_1, r_2, \dots, r_N)$. The raw basis generator then allows differentiated factors from several tensor species, while the IBP and FP are still performed on the scalar ansatz exactly as before.

For field redefinitions, one must compute the image of the free equations of motion of each field. Schematically, if the free quadratic action is $L_0[\phi]$, a general local redefinition $\phi^a \rightarrow \phi^a + \delta\phi^a$ induces

$$\Delta L = \sum_a E_a \delta\phi^a + \text{total derivative},$$

where $E_a = \delta L_0 / \delta \phi^a$ are the free equations of motion. In practice this means building a redefinition basis for each field species, computing its image under the free quadratic action, and combining all projected image vectors before quotienting the sector basis.

9. Practical caveats

- Validate the first nontrivial sectors on your installation before launching a long run.
- For large dimensions, raw contraction generation can grow rapidly: use `SectorList` to target only sectors needed in the paper.
- Keep derivative order and field content explicit in cache keys so that changing conventions does not silently reuse incompatible files.
- When interpreting the final basis physically, remember that the script removes redundancies relative

March 2026

Contents

1	Purpose and scope	2
2	Mathematical backbone of the algorithm	2
2.1	Sector decomposition	2
2.2	IBP via the Euler operator	2
2.3	Fierz–Pauli field redefinitions	3
3	Toolkit architecture	3
3.1	What the sector data contain	3
4	Running the script	4
4.1	Why the cache matters	4
5	Extracting the required field redefinition	4
6	Extension to multi-field theories	5
7	Recommended validation sequence	5
8	Full script listing	6
9	Deliverables and associated files	15

1 Purpose and scope

The toolkit in `graviton_basis_toolkit.wl` is meant to automate the following task. Given a symmetric rank-2 tensor field $h_{\mu\nu}$ on flat space, enumerate all local scalar operators built from some number of gravitons and derivatives, then reduce them in two stages:

1. quotient by integration by parts (IBP), and
2. quotient by field redefinitions induced by the quadratic Fierz–Pauli action.

The key improvement over the earlier one-sector workflow is that the script is organized by sectors

$$D = n_h + N_\partial, \quad (1)$$

where n_h is the number of graviton fields and N_∂ is the total number of derivatives. Running to a higher maximum dimension therefore reuses all previously computed lower-dimensional sectors.

The toolkit is not specialized to the cubic two-derivative sector. It loops over all scalar graviton sectors up to a chosen `MaxDimension`, computes a raw basis in each sector, reduces it modulo IBP, and then quotients the result by the image of admissible Fierz–Pauli field redefinitions.

2 Mathematical backbone of the algorithm

2.1 Sector decomposition

For the linearized graviton, a scalar monomial with n_h fields and N_∂ derivatives belongs to the sector

$$\mathcal{V}_{n_h, N_\partial}. \quad (2)$$

The script generates all Lorentz contractions in this sector by first building all derivative partitions of N_∂ among the n_h graviton factors and then applying `AllContractions` to each seed monomial.

For example, in the mass-dimension-five cubic derivative sector one has

$$(n_h, N_\partial) = (3, 2), \quad \mathcal{V}_{3,2} = \text{span}\{h(\partial h)(\partial h), hh(\partial\partial h)\}. \quad (3)$$

The algorithm does not assume a preferred derivative placement; the IBP quotient handles all relations after the raw basis is generated.

2.2 IBP via the Euler operator

The criterion for a local scalar density X to be IBP-trivial is the vanishing of its Euler–Lagrange derivative:

$$X \sim 0 \mod \text{IBP} \iff \mathcal{E}^{\mu\nu}(X) = 0. \quad (4)$$

For a basis $\{O_i\}$ and ansatz

$$X = \sum_i c_i O_i, \quad (5)$$

one solves

$$\mathcal{E}^{\mu\nu}(X) = 0 \quad (6)$$

for the coefficients c_i . The nullspace gives the total derivatives, while deleting pivot columns produces a basis of the quotient space $\mathcal{V}_{n_h, N_\partial}/\text{IBP}$.

Operationally, this is what the script stores in the entries

$$\text{"IBPRelationMatrix"}, \quad \text{"IBPBasis"}, \quad \text{"IBPCount"}. \quad (7)$$

2.3 Fierz–Pauli field redefinitions

The second quotient uses the quadratic Fierz–Pauli Lagrangian

$$\mathcal{L}_{\text{FP}}^{(2)} = -\frac{1}{2}\partial_\rho h_{\mu\nu}\partial^\rho h^{\mu\nu} + \partial_\mu h^{\mu\nu}\partial^\rho h_{\rho\nu} - \partial_\mu h^{\mu\nu}\partial_\nu h + \frac{1}{2}\partial_\rho h\partial^\rho h. \quad (8)$$

Its equations of motion carry one power of h and two derivatives. Therefore, to contribute to a target scalar sector (n_h, N_∂) , the relevant quadratic-order graviton field redefinitions have the schematic form

$$\Delta h_{\mu\nu} \sim (\partial)^{N_\partial-2} h^{n_h-1}. \quad (9)$$

This is the rule built into the toolkit: for each target sector it constructs the appropriate rank-2 basis for $\Delta h_{\mu\nu}$, computes the first-order shift of the Fierz–Pauli action, and projects the resulting images into the IBP quotient.

For a target sector (n_h, N_∂) , the field-redefinition problem is itself sector-local: one only needs the rank-2 basis with $(n_h - 1)$ graviton factors and $(N_\partial - 2)$ derivatives.

3 Toolkit architecture

The file is organized around a small number of high-level building blocks.

Function or block	Purpose
<code>RawScalarBasis[nh,Nd]</code>	Generates and caches all canonical scalar contractions in the sector $(n_h, N_\partial) = (\text{nh}, \text{Nd})$.
<code>IBPData[basis]</code>	Computes the Euler-operator relations, free parameters, and relation matrix for the supplied raw basis.
<code>QuotientBasisFromRelation</code>	Deletes pivot columns and returns a basis modulo IBP.
<code>Rank2Basis[nhRedef,NdRedef]</code>	Generates the symmetric rank-2 basis used for $\Delta h_{\mu\nu}$.
<code>ShiftFromRedefExpr</code>	Computes the first-order shift of the quadratic Fierz–Pauli Lagrangian under $h \rightarrow h + \lambda \Delta h$.
<code>ProjectModuloIBP</code>	Projects an expression into the same IBP quotient basis by requiring equality up to total derivatives.
<code>RedefinitionImageData</code>	Packages the projected field-redefinition images, their coordinate vectors, and an independent spanning subset.
<code>ComputeSectorData</code>	Runs the entire raw $\rightarrow$ IBP $\rightarrow$ FP-redefinition pipeline for a single sector and caches the result.
<code>RunBasisComputation</code>	Main driver that loops through all sectors up to <code>MaxDimension</code> or only a user-provided subset.

3.1 What the sector data contain

Each sector is stored as an association with entries such as

$$\text{"RawBasis"}, \quad \text{"RawCount"}, \quad (10)$$

$$\text{"IBPBasis"}, \quad \text{"IBPCount"}, \quad (11)$$

$$\text{"RedefIndependentDeltas"}, \quad \text{"RedefRank"}, \quad (12)$$

$$\text{"PhysicalBasis"}, \quad \text{"PhysicalCount"}. \quad (13)$$

This makes it easy to inspect intermediate stages as well as the final quotient basis.

4 Running the script

A standard run up to mass dimension six looks like

```
res = RunBasisComputation[
  "MaxDimension" -> 6,
  "CacheDirectory" -> FileNameJoin[{NotebookDirectory[], "basis_cache"}],
  "UseCache" -> True,
  "ForceRecompute" -> False,
  "Verbose" -> True,
  "ExportSummaryCSV" -> True
];
```

If only a specific sector is needed, the run can be restricted explicitly:

```
res32 = RunBasisComputation[
  "SectorList" -> {{3, 2}},
  "CacheDirectory" -> FileNameJoin[{NotebookDirectory[], "basis_cache"}],
  "UseCache" -> True
];
```

Useful post-processing calls are

```
PrintSectorSummary[res]
PhysicalBasisOfSector[res, 3, 2]
IBPBasisOfSector[res, 3, 2]
RedefinitionRulesOfSector[res, 3, 2]
```

4.1 Why the cache matters

Suppose one first computes all sectors up to $D = 5$ and later reruns up to $D = 6$. Since the script caches each sector separately, all previously computed sectors are loaded from disk and only the new $D = 6$ sectors are evaluated. This is particularly valuable because the number of raw contractions and the size of the linear systems typically grow with D .

The cache is written sector by sector, with file names that encode (n_h, N_∂) and related redefinition-sector data. This is more robust than caching only a final combined result because it keeps lower-dimensional work reusable even if a later sector fails or if the user decides to rerun only part of the calculation.

5 Extracting the required field redefinition

The script also returns the independent rank-2 tensors whose images span the removable directions in the IBP quotient. This is the raw material needed to reconstruct an explicit field redefinition.

In linear-algebra terms, let $\{B_i\}$ be the IBP basis and let the independent projected redefinition images have coordinate vectors collected into a matrix I . A general interaction written as

$$\mathcal{L}_{\text{int}} = \sum_i c_i B_i \quad (14)$$

can be reduced to the chosen physical basis by solving for coefficients α_r such that

$$c + I^T \alpha \quad (15)$$

has no support on the pivot directions removed by field redefinitions. The same logic can be used either to remove a single pivot operator or to reduce an arbitrary coupling vector to the chosen quotient basis.

The practical output to inspect is

$$\text{result}["\text{SectorData}"][\text{Key}[\text{nh}, \text{Nd}]]["\text{RedefIndependentDeltas}"]. \quad (16)$$

These are the elementary $\Delta h_{\mu\nu}$ tensors that span the image. One can then solve the corresponding finite linear system to obtain the explicit combined field redefinition.

6 Extension to multi-field theories

The single-field logic generalizes cleanly. If the quadratic theory contains fields

$$\Phi^A = \{h_{\mu\nu}, \phi, A_\mu, \dots\}, \quad (17)$$

then the correct IBP criterion is the simultaneous vanishing of all Euler derivatives,

$$X \sim 0 \pmod{\text{IBP}} \iff \mathcal{E}_A(X) = 0 \text{ for every field } \Phi^A. \quad (18)$$

Similarly, the first-order change under field redefinitions is

$$\Delta \mathcal{L}_0 = \sum_A E_A \Delta^A + \partial_\mu K^\mu, \quad (19)$$

so the image matrix must include redefinitions of all fields that can feed the target interaction sector.

In practice, the multi-field extension requires four modifications:

1. build the raw basis in the desired mixed sector,
2. impose the all-field Euler conditions for IBP,
3. generate admissible redefinition families for each field,
4. shift the full quadratic Lagrangian and project all images into one common IBP quotient.

The same matrix-rank and pivot-column logic then produces the final basis.

7 Recommended validation sequence

Before using the toolkit for a large scan, it is sensible to validate it in increasing stages:

1. run only the known $(3, 2)$ sector and confirm the expected raw and IBP counts,
2. inspect the independent redefinition image vectors and their rank,
3. run all sectors up to $D = 5$ with caching enabled,
4. extend to $D = 6$ and confirm that the lower sectors are loaded from cache,
5. only then move on to multi-field sectors or higher dimensions.

The most reliable way to use the toolkit is to treat each layer as a separate correctness test: raw basis generation first, IBP quotient second, and field-redefinition quotient third.

8 Full script listing

For completeness, the full single-file toolkit used by the report is included below.

```
(* ::Package:: *)

(* ===== *)
(* Graviton operator basis toolkit *)
(* - raw scalar basis *)
(* - quotient by IBP *)
(* - quotient by Fierz-Pauli field redefinitions *)
(* - sector-level caching *)
(* - optional restriction to selected sectors *)
(* - summary export *)
(* - extraction of redefinitions spanning the quotient image *)
(* ===== *)

Needs["xAct"xTras"];

(* ===== *)
(* 0. Public entry points / options *)
(* ===== *)

ClearAll[
  RunBasisComputation, ComputeSectorData, LoadOrCompute,
  SetCacheDirectory, ClearCacheDirectory,
  ExportSummaryCSV, ExportSummaryMX,
  RawBasisOfSector, IBPBasisOfSector, PhysicalBasisOfSector,
  RedefinitionsOfSector, RedefinitionRulesOfSector,
  PrintSectorSummary, SectorSummaryTable
];

Options[RunBasisComputation] = {
  "MaxDimension" -> 5,
  "SectorList" -> Automatic,
  "CacheDirectory" -> Automatic,
  "UseCache" -> True,
  "ForceRecompute" -> False,
  "Verbose" -> True,
  "ExportSummaryCSV" -> False,
  "SummaryCSVFile" -> Automatic,
  "ExportResultMX" -> False,
  "ResultMXFile" -> Automatic
};

$GravitonBasisCacheDirectory = Automatic;

SetCacheDirectory[dir_String] := ($GravitonBasisCacheDirectory = dir);
ClearCacheDirectory[] := ($GravitonBasisCacheDirectory = Automatic);

(* ===== *)
(* 1. xAct setup *)
(* ===== *)

Quiet @ Check[
  DefManifold[M, 4, IndexRange[{a, g}, {i, z}]],
  Null
];

Quiet @ Check[
  DefMetric[-1, eta[-a, -b], PD, FlatMetric -> True,
    SymbolOfCovD -> {"", "\[PartialD]"},
    Null
];

Quiet @ Check[
  DefTensor[h[-a, -b], M, Symmetric[{-a, -b}], PrintAs -> "h"],
```

```

    Null
];

Quiet @ Check[
  DefTensor[dh[-a, -b], M, Symmetric[{-a, -b}], PrintAs -> "\[Delta]h"],
  Null
];

Quiet @ Check[
  DefTensor[probe[-a, -b], M, Symmetric[{-a, -b}], PrintAs -> "P"],
  Null
];

(* constant substitution parameter for field redefinitions *)
Quiet @ Check[
  DefConstantSymbol[lam],
  Null
];

(* ===== *)
(* 2. Utilities *)
(* ===== *)

ClearAll[
  canonExpr, canonList, PivotColumns, PivotRows, LinearCombination,
  CompositionList, CacheDirectoryResolved, CacheFile,
  SaveExpr, LoadExpr, PrettyKey, LogPrint,
  SectorKeyAssociation, NormalizeSectorList,
  SafeSolveConstants, ZeroVectorQ, CoordinatesMatrixQ
];

canonExpr[expr_] := CollectTensors @ ToCanonical @ ContractMetric @ Expand[expr];
canonList[list_List] := DeleteDuplicates @ (canonExpr /@ list);

PivotColumns[m_?MatrixQ] := Module[{rr},
  rr = RowReduce[m];
  Cases[
    (FirstPosition[#, _?(# != 0 &), Nothing, {1}, Heads -> False] & /@ rr),
    {j_} :> j
  ]
];

PivotRows[m_?MatrixQ] := PivotColumns[Transpose[m]];

LinearCombination[vec_List, basis_List] /; Length[vec] == Length[basis] :=
  Total @ MapThread[Times, {vec, basis}];

CompositionList[n_Integer, k_Integer] /; n >= 0 && k >= 1 := Compositions[n, k];

SectorKeyAssociation[nh_Integer, Nd_Integer] := <|"nh" -> nh, "Nd" -> Nd, "D" -> nh + Nd|>;

NormalizeSectorList[Automatic, dmax_Integer] :=
  Flatten[Table[{nh, d - nh}, {d, 1, dmax}, {nh, 1, d}], 1];
NormalizeSectorList[list_List, _Integer] := DeleteDuplicates @ Select[list, MatchQ[#, {_Integer, _Integer}]
  &];

CacheDirectoryResolved[opts : OptionsPattern[RunBasisComputation]] := Module[{dir},
  dir = OptionValue["CacheDirectory"];
  Which[
    dir === Automatic && $GravitonBasisCacheDirectory != Automatic, $GravitonBasisCacheDirectory,
    dir === Automatic,
      FileNameJoin[{NotebookDirectory[] /. $Failed -> Directory[], "graviton_basis_cache"}],
    True, dir
  ]
];

PrettyKey[assoc_Association] := StringRiffle[
  KeyValueMap[ToString[#1] <> "_" <> ToString[#2] &, assoc],

```

```

    "--"
];

CacheFile[type_String, key_Association, opts : OptionsPattern[RunBasisComputation]] := Module[{dir},
  dir = CacheDirectoryResolved[opts];
  If[! DirectoryQ[dir], CreateDirectory[dir, CreateIntermediateDirectories -> True]];
  FileNameJoin[{dir, type <> "--" <> PrettyKey[key] <> ".m"}]
];

SaveExpr[file_String, expr_] := Put[expr, file];
LoadExpr[file_String] := Get[file];

LogPrint[msg_, opts : OptionsPattern[RunBasisComputation]] :=
  If[TrueQ[OptionValue["Verbose"]], Print[msg]];

LoadOrCompute[type_String, key_Association, compute_, opts : OptionsPattern[RunBasisComputation]] := Module[
  {file, useCache, force, result},
  file = CacheFile[type, key, opts];
  useCache = TrueQ[OptionValue["UseCache"]];
  force = TrueQ[OptionValue["ForceRecompute"]];

  If[useCache && FileExistsQ[file] && ! force,
    LogPrint["[cache] loading " <> type <> " : " <> PrettyKey[key], opts];
    Return[LoadExpr[file]];
  ];

  LogPrint["[compute] " <> type <> " : " <> PrettyKey[key], opts];
  result = compute[];

  If[useCache, SaveExpr[file, result]];
  result
];

SafeSolveConstants[eq_] := Module[{sol},
  sol = SolveConstants[eq];
  Which[
    sol === {}, {{}},
    ListQ[sol], sol,
    True, {{}}
  ]
];

ZeroVectorQ[v_List] := And @@ (TrueQ[# === 0] & /@ v);
CoordinatesMatrixQ[x_] := MatrixQ[x] || x === {};

(* ===== *)
(* 3. Fierz--Pauli quadratic Lagrangian *)
(* ===== *)

ClearAll[LFP];

LFP = canonExpr[
  -1/2 PD[a] @ h[b, c] PD[-a] @ h[-b, -c]
  + PD[a] @ h[-a, b] PD[c] @ h[-c, -b]
  - PD[a] @ h[-a, b] PD[-b] @ h[c, -c]
  + 1/2 PD[a] @ h[b, -b] PD[-a] @ h[c, -c]
];

(* ===== *)
(* 4. Raw monomials and bases *)
(* ===== *)

ClearAll[
  MakeFieldWithKDerivs, ScalarMonomialFromPartition,
  ProbeMonomialFromPartition, RawScalarMonomials,
  RawProbeMonomials, RawScalarBasis, Rank2Basis
];

```

```

MakeFieldWithKDerivs[tensor_Symbol, k_Integer] := Module[{mu, nu, ders, expr},
  mu = DummyIn[TangentM];
  nu = DummyIn[TangentM];
  ders = Table[DummyIn[TangentM], {k}];
  expr = tensor[mu, nu];
  Do[expr = PD[-ders[[j]]] @ expr, {j, k}];
  expr
];

ScalarMonomialFromPartition[part_List] := Times @@ (MakeFieldWithKDerivs[h, #] & /@ part);
ProbeMonomialFromPartition[part_List] := Module[{A, B},
  A = DummyIn[TangentM];
  B = DummyIn[TangentM];
  probe[A, B] Times @@ (MakeFieldWithKDerivs[h, #] & /@ part)
];

RawScalarMonomials[nh_Integer, Nd_Integer] := ScalarMonomialFromPartition /@ CompositionList[Nd, nh];
RawProbeMonomials[nh_Integer, Nd_Integer] := ProbeMonomialFromPartition /@ CompositionList[Nd, nh];

RawScalarBasis[nh_Integer, Nd_Integer, opts : OptionsPattern[RunBasisComputation]] :=
  LoadOrCompute[
    "raw_scalar_basis",
    <|"nh" -> nh, "Nd" -> Nd|>,
    Function[Null,
      canonList @ Flatten[AllContractions /@ RawScalarMonomials[nh, Nd]]
    ],
    opts
  ];

(* rank-2 basis used for field redefinitions delta h_{uv} *)
Rank2Basis[nhRedef_Integer, NdRedef_Integer, opts : OptionsPattern[RunBasisComputation]] :=
  LoadOrCompute[
    "rank2_basis",
    <|"nhRedef" -> nhRedef, "NdRedef" -> NdRedef|>,
    Function[Null,
      Module[{probeScalars, basisExprs},
        If[nhRedef < 1 || NdRedef < 0, Return[{}]];
        probeScalars = canonList @ Flatten[AllContractions /@ RawProbeMonomials[nhRedef, NdRedef]];
        basisExprs = canonList @ (canonExpr @ VarD[probe[-u, -v], PD][#] & /@ probeScalars);
        basisExprs
      ]
    ],
    opts
  ];

(* ===== *)
(* 5. IBP quotient *)
(* ===== *)

ClearAll[IBPData, QuotientBasisFromRelations];

IBPData[basis_List] := Module[
  {ans, EL, sol, allc, freec, relMat, relExprs},

  If[basis == {},
    Return[<|
      "Solution" -> {},
      "FreeParameters" -> {},
      "RelationMatrix" -> {},
      "RelationExpressions" -> {}
    |>]
  ];

  ans = MakeAnsatz[basis, ConstantPrefix -> c];
  EL = canonExpr @ VarD[h[-m, -n], PD][ans];
  sol = SafeSolveConstants[EL == 0];

  allc = Table[Symbol["c" <> ToString[i]], {i, Length[basis]}];

```

```

freec = DeleteDuplicates @ Cases[
  Last /@ First[sol],
  s_Symbol /; StringMatchQ[SymbolName[s], "c" ~~ DigitCharacter ..],
  Infinity
];

relMat = If[freec === {}, {},
  Table[
    allc /. First[sol] /. Thread[freec -> UnitVector[Length[freec], j]],
    {j, Length[freec]}
  ]
];

relExprs = If[relMat === {}, {}, canonExpr /@ (LinearCombination[#, basis] & /@ relMat)];

<|
  "Solution" -> sol,
  "FreeParameters" -> freec,
  "RelationMatrix" -> relMat,
  "RelationExpressions" -> relExprs
|>
];

QuotientBasisFromRelations[basis_List, relMat_] := Module[{pivot},
  pivot = If[MatrixQ[relMat] && relMat != {}, PivotColumns[relMat], {}];
  <|
    "PivotColumns" -> pivot,
    "Basis" -> Delete[basis, List /@ Sort[pivot]]
  |>
];

(* ===== *)
(* 6. Field-redefinition image *)
(* ===== *)

ClearAll[
  ShiftFromRedefExpr, ProjectModuloIBP,
  IndependentImageRows, BasisCoordinatesFromIndependentRows,
  RedefinitionImageData
];

(* IMPORTANT:
  Expand the FP Lagrangian with h -> h + lam dh, extract the coefficient of lam,
  and only then replace dh by a concrete rank-2 tensor deltaExpr[u,v].
  lam is a DefConstantSymbol, so PD[lam] = 0 automatically. *)
ShiftFromRedefExpr[deltaExpr_] := Module[{expr},
  expr = Expand[LFP /. HoldPattern[h[x_, y_]] :> (h[x, y] + lam dh[x, y])];
  expr = Coefficient[expr, lam, 1];
  expr = expr /. HoldPattern[dh[x_, y_]] :> (deltaExpr /. {u -> x, v -> y});
  canonExpr[expr]
];

ProjectModuloIBP[expr_, basis_List] := Module[{ans, eq, sol, qvars, coeffs},
  If[basis === {},
    Return[<| "ProjectedExpr" -> 0, "Coordinates" -> {}, "Solution" -> {} |>]
  ];

  ans = MakeAnsatz[basis, ConstantPrefix -> q];
  qvars = Table[Symbol["q" <> ToString[i]], {i, Length[basis]}];
  eq = canonExpr @ VarD[h[-m, -n], PD][ans - expr];
  sol = SafeSolveConstants[eq == 0];
  coeffs = qvars /. First[sol];

  <|
    "ProjectedExpr" -> canonExpr[ans /. First[sol]],
    "Coordinates" -> coeffs,
    "Solution" -> sol
  |>
];

```

```

|>
];

IndependentImageRows[vecs_] := Module[{rows},
  If[! MatrixQ[vecs] || vecs === {}, Return[{}]];
  rows = PivotRows[vecs];
  rows
];

BasisCoordinatesFromIndependentRows[indVecs_?MatrixQ] := Module[{piv},
  piv = PivotColumns[indVecs];
  <|"PivotColumns" -> piv|>
];

BasisCoordinatesFromIndependentRows[{}] := <|"PivotColumns" -> {}|>;

RedefinitionImageData[nh_Integer, Nd_Integer, ibpBasis_List, opts : OptionsPattern[RunBasisComputation]] :=
  LoadOrCompute[
    "redef_image_data",
    <|"nh" -> nh, "Nd" -> Nd|>,
    Function[Null,
      Module[{nhRedef, NdRedef, deltaBasis, shifts, proj, vecs, indRows, indVecs},

        nhRedef = nh - 1;
        NdRedef = Nd - 2;

        If[nhRedef < 1 || NdRedef < 0 || ibpBasis === {},
          Return[<|
            "DeltaBasis" -> {},
            "ShiftImages" -> {},
            "ProjectedData" -> {},
            "ImageVectors" -> {},
            "IndependentRows" -> {},
            "IndependentVectors" -> {},
            "IndependentDeltas" -> {},
            "Rank" -> 0
          |>]
        ];

        deltaBasis = Rank2Basis[nhRedef, NdRedef, opts];
        shifts = ShiftFromRedefExpr /@ deltaBasis;
        proj = ProjectModuloIBP[#, ibpBasis] & /@ shifts;
        vecs = proj[[All, "Coordinates"]];

        indRows = IndependentImageRows[vecs];
        indVecs = If[indRows === {}, {}, vecs[[indRows]]];

        <|
          "DeltaBasis" -> deltaBasis,
          "ShiftImages" -> shifts,
          "ProjectedData" -> proj,
          "ImageVectors" -> vecs,
          "IndependentRows" -> indRows,
          "IndependentVectors" -> indVecs,
          "IndependentDeltas" -> If[indRows === {}, {}, deltaBasis[[indRows]]],
          "Rank" -> If[MatrixQ[vecs] && vecs != {}, MatrixRank[vecs], 0]
        |>
      ]
    ],
    opts
  ];

(* ===== *)
(* 7. Extract explicit redefinitions selecting the quotient image *)
(* ===== *)

ClearAll[RedefinitionRulesFromVectors];

RedefinitionRulesFromVectors[ibpBasis_List, indDeltas_List, indVecs_] := Module[

```

```

{rules},
If[indDeltas === {} || indVecs === {} || ! MatrixQ[indVecs],
  Return[{}]]
];

rules = Table[
  <|
    "Delta" -> indDeltas[[i]],
    "ImageCoordinatesInIBPBasis" -> indVecs[[i]],
    "ImageExpressionInIBPBasis" -> canonExpr @ LinearCombination[indVecs[[i]], ibpBasis]
  |>,
  {i, Length[indDeltas]}
];

rules
];

(* ===== *)
(* 8. Full sector computation *)
(* ===== *)

ClearAll[ComputeSectorData];

ComputeSectorData[nh_Integer, Nd_Integer, opts : OptionsPattern[RunBasisComputation]] :=
  LoadOrCompute[
    "sector_data",
    SectorKeyAssociation[nh, Nd],
    Function[Null,
      Module[{rawBasis, ibp, qbasis, redef, piv, physBasis, redefRules},

        rawBasis = RawScalarBasis[nh, Nd, opts];
        ibp = IBPData[rawBasis];
        qbasis = QuotientBasisFromRelations[rawBasis, ibp["RelationMatrix"]];
        redef = RedefinitionImageData[nh, Nd, qbasis["Basis"], opts];

        piv = If[MatrixQ[redef["IndependentVectors"]] && redef["IndependentVectors"] != {},
          PivotColumns[redef["IndependentVectors"]],
          {}
        ];

        physBasis = Delete[qbasis["Basis"], List /@ Sort[piv]];
        redefRules = RedefinitionRulesFromVectors[
          qbasis["Basis"],
          redef["IndependentDeltas"],
          redef["IndependentVectors"]
        ];

        <|
          "nh" -> nh,
          "Nd" -> Nd,
          "Dimension" -> nh + Nd,

          "RawBasis" -> rawBasis,
          "RawCount" -> Length[rawBasis],

          "IBPSolution" -> ibp["Solution"],
          "IBPFreeParameters" -> ibp["FreeParameters"],
          "IBPRelationMatrix" -> ibp["RelationMatrix"],
          "IBPRelationExpressions" -> ibp["RelationExpressions"],
          "IBPPivots" -> qbasis["PivotColumns"],
          "IBPBasis" -> qbasis["Basis"],
          "IBPCount" -> Length[qbasis["Basis"]],

          "RedefDeltaBasis" -> redef["DeltaBasis"],
          "RedefShiftImages" -> redef["ShiftImages"],
          "RedefProjectedData" -> redef["ProjectedData"],
          "RedefImageVectors" -> redef["ImageVectors"],
          "RedefIndependentRows" -> redef["IndependentRows"],

```

```

    "RedefIndependentVectors" -> redef["IndependentVectors"],
    "RedefIndependentDeltas" -> redef["IndependentDeltas"],
    "RedefRank" -> redef["Rank"],
    "RedefPivots" -> piv,
    "RedefinitionRules" -> redefRules,

    "PhysicalBasis" -> physBasis,
    "PhysicalCount" -> Length[physBasis]
  ]>
  ],
  opts
];

(* ===== *)
(* 9. Exports and summaries *)
(* ===== *)

ClearAll[SectorSummaryTable, PrintSectorSummary, ExportSummaryCSV, ExportSummaryMX];

SectorSummaryTable[result_Association] := Module[{sum, keys},
  sum = result["Summary"];
  keys = Keys[sum];
  Table[
    {
      key[[1]], key[[2]],
      sum[key]["Dimension"],
      sum[key]["RawCount"],
      sum[key]["IBPCount"],
      sum[key]["RedefRank"],
      sum[key]["PhysicalCount"]
    },
    {key, keys}
  ]
];

PrintSectorSummary[result_Association] := Module[{tbl},
  tbl = SectorSummaryTable[result];
  Print @ Grid[
    Prepend[tbl, {"nh", "Nd", "D", "RawCount", "IBPCount", "RedefRank", "PhysicalCount"}],
    Frame -> All
  ];
];

ExportSummaryCSV[result_Association, file_String] := Module[{tbl},
  tbl = SectorSummaryTable[result];
  Export[file,
    Prepend[tbl, {"nh", "Nd", "D", "RawCount", "IBPCount", "RedefRank", "PhysicalCount"}],
    "CSV"
  ];
];

ExportSummaryMX[result_Association, file_String] := Export[file, result, "MX"];

(* ===== *)
(* 10. Convenience accessors *)
(* ===== *)

ClearAll[
  RawBasisOfSector, IBPBasisOfSector, PhysicalBasisOfSector,
  RedefinitionsOfSector, RedefinitionRulesOfSector
];

RawBasisOfSector[result_Association, nh_Integer, Nd_Integer] :=
  result["SectorData"][Key[{nh, Nd}]]["RawBasis"];

IBPBasisOfSector[result_Association, nh_Integer, Nd_Integer] :=
  result["SectorData"][Key[{nh, Nd}]]["IBPBasis"];

```



```

PhysicalBasisOfSector[result_Association, nh_Integer, Nd_Integer] :=
  result["SectorData"][Key[{nh, Nd}]]["PhysicalBasis"];

RedefinitionsOfSector[result_Association, nh_Integer, Nd_Integer] :=
  result["SectorData"][Key[{nh, Nd}]]["RedefIndependentDeltas"];

RedefinitionRulesOfSector[result_Association, nh_Integer, Nd_Integer] :=
  result["SectorData"][Key[{nh, Nd}]]["RedefinitionRules"];

(* ===== *)
(* 11. Master driver *)
(* ===== *)

RunBasisComputation[opts : OptionsPattern[]] := Module[
  {dmax, sectors, data, summary, result, csvFile, mxFile},

  dmax = OptionValue["MaxDimension"];
  sectors = NormalizeSectorList[OptionValue["SectorList"], dmax];
  sectors = Select[sectors, #[[1]] >= 1 && #[[2]] >= 0 && Total[#] <= dmax &];

  data = Association @ Table[
    With[{nh = sec[[1]], Nd = sec[[2]]},
      Key[{nh, Nd}] -> ComputeSectorData[nh, Nd, opts]
    ],
    {sec, sectors}
  ];

  summary = Association @ Table[
    With[{d = data[Key[sec]]},
      Key[sec] -> <|
        "Dimension" -> d["Dimension"],
        "RawCount" -> d["RawCount"],
        "IBPCount" -> d["IBPCount"],
        "RedefRank" -> d["RedefRank"],
        "PhysicalCount" -> d["PhysicalCount"]
      |>
    ],
    {sec, sectors}
  ];

  result = <|
    "MaxDimension" -> dmax,
    "SectorList" -> sectors,
    "SectorData" -> data,
    "Summary" -> summary
  |>;

  If[TrueQ[OptionValue["ExportSummaryCSV"]],
    csvFile = OptionValue["SummaryCSVFile"];
    If[csvFile === Automatic,
      csvFile = FileNameJoin[{CacheDirectoryResolved[opts], "graviton_basis_summary.csv"}]
    ];
    ExportSummaryCSV[result, csvFile];
    LogPrint["[export] summary CSV -> " <> csvFile, opts];
  ];

  If[TrueQ[OptionValue["ExportResultMX"]],
    mxFile = OptionValue["ResultMXFile"];
    If[mxFile === Automatic,
      mxFile = FileNameJoin[{CacheDirectoryResolved[opts], "graviton_basis_result.mx"}]
    ];
    ExportSummaryMX[result, mxFile];
    LogPrint["[export] result MX -> " <> mxFile, opts];
  ];

  result
];

```

```

(* ===== *)
(* 12. Example usage *)
(* ===== *)

(**

res5 = RunBasisComputation[
  "MaxDimension" -> 5,
  "CacheDirectory" -> FileNameJoin[{NotebookDirectory[], "basis_cache"}],
  "UseCache" -> True,
  "ForceRecompute" -> False,
  "Verbose" -> True,
  "ExportSummaryCSV" -> True,
  "ExportResultMX" -> True
];

PrintSectorSummary[res5];

(* the sector relevant to dim-5 cubic derivative terms *)
phys32 = PhysicalBasisOfSector[res5, 3, 2];
phys32 // TableForm;

(* independent field redefinitions that span the quotient image *)
redef32 = RedefinitionRulesOfSector[res5, 3, 2];
redef32 // TableForm;

(* rerun to D=6 reusing D<=5 cache *)
res6 = RunBasisComputation[
  "MaxDimension" -> 6,
  "CacheDirectory" -> FileNameJoin[{NotebookDirectory[], "basis_cache"}],
  "UseCache" -> True,
  "ForceRecompute" -> False,
  "Verbose" -> True
];

(* restrict to a single sector if desired *)
resSingle = RunBasisComputation[
  "MaxDimension" -> 8,
  "SectorList" -> {{3, 2}},
  "CacheDirectory" -> FileNameJoin[{NotebookDirectory[], "basis_cache"}],
  "UseCache" -> True
];

***)

```

9 Deliverables and associated files

The report accompanies the following files:

- graviton_basis_toolkit.wl: the full Mathematica/xAct toolkit,
- graviton_basis_companion_note.pdf: the shorter operational note,
- operator_basis_toolkit_report.pdf: this polished report-format companion.